```
    if (empty($in)) {
        exit("And we're done!" . PHP_EOL);
    }
}

function mainLoop(): never
{
    while (true) {
        shallWeContinue();
        print 'Hey ho, on we go...' . PHP_EOL;
    }
}

mainLoop();

Output:
Type something to continue, or press Enter on its own to abort
go for it
Hey ho, on we go...
Type something to continue, or press Enter on its own to abort
And we're done!
```

`shallWeContinue()` does the job of asking for user input (from a command line prompt, via the `readline()` call), and handling the typed response. An empty user response results in a call to `exit()`, which immediately terminates the program – the end of the function scope (the closing brace, if you remember) is never reached. The use of the infinite looping `while (true) { }` construct means that `mainLoop()` can never return, and a good IDE should detect this infinite loop (or some other non-returnable context like a program-terminating statement) or otherwise highlight that the `never` return type is invalid. Let's tweak our code so that we break the `while` loop rather than exiting the program completely and see what happens:

```
function shallWeContinue(): bool
{
    $in = readline('Type something to continue, or press Enter on its own
    to abort' . PHP_EOL);

    return (!empty($in));
}
```